Rapport

I) Dijkstra

Les différents tests ont été réalisés sur Linux avec un ordinateur possédant 16 processeurs, 8 cœurs physiques, et 16 cœurs logiques.

1) Stratégies de parallélisation

Dans le programme, il est possible de paralléliser différentes parties : la fonction de calcul dans le main, et les deux fonctions présentes dans le programme : Dijkstra et createRandomGraph.

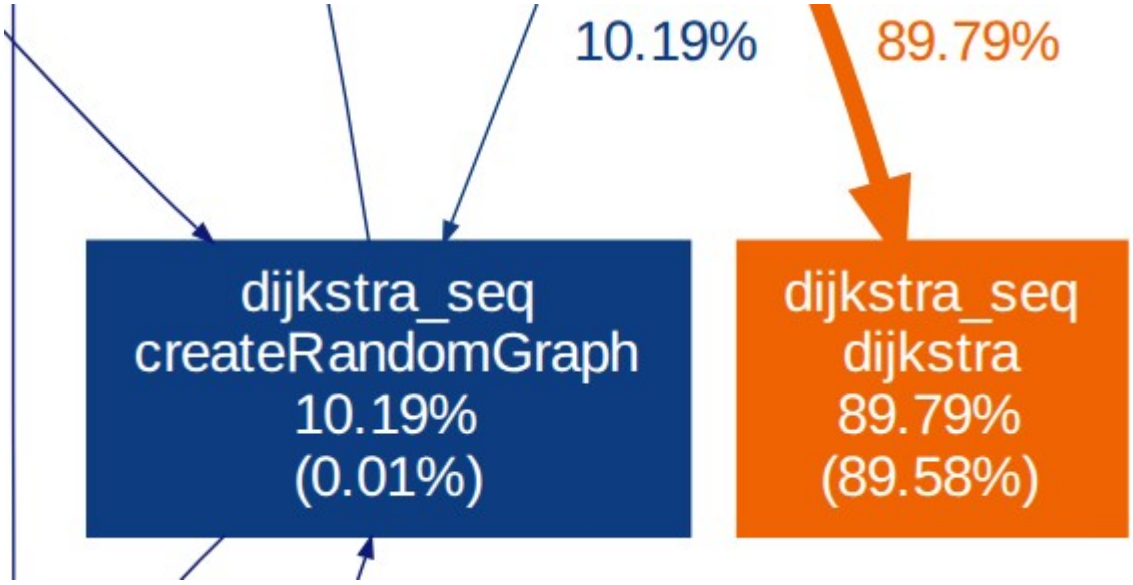
1.1) Technologies

Étant donné que le programme traite un seul et unique graphe, l'utilisation de la bibliothèque OpenMP semble la plus efficace par rapport à MPI. En effet, en utilisant l'algorithme actuel, il n'est pas possible de découper le graphe pour en traiter certaines parties : il y a une logique, séquentielle, à suivre qui pourra cependant profiter de la parallélisation dans ses différentes sous-parties. Si le programme traitait plusieurs graphes, l'utilisation des deux bibliothèques pourrait être mis en place.

1.2) Le parallélisme

1.2.1) Programme séquentiel

Tout d'abord, il faut regarder quelles parties du programme seront les plus intéressantes à paralléliser.



Dans l'état actuel, la fonction prenant le plus de temps à s'exécuter est Dijkstra qui prend environ 89,79% du temps. La deuxième fonction, createRandomGraph, ne représente

que 10,19% du temps : paralléliser cette fonction n’améliorera donc probablement que très peu les performances du programme.

Pour avoir un même référentiel pendant les différentes améliorations du code, exécutons le programme avec en entrée 200 000 sommets, 100 000 arêtes, et une graine de 100. Avec le fag -O2 lors de la compilation, et 16 threads utilisés, le résultat obtenu est 2147472909.58 avec une moyenne de 18,162 secondes d’exécution et un écart-type de 0,192.

1.2.2) Parallélisation du main

La parallélisation du calcul du résultat peut se réaliser simplement avec une réduction de la variable contenant le résultat. En utilisant le même référentiel décrit précédemment, le programme donne le bon résultat avec une moyenne de 18,169 secondes d’exécution et un écart-type de 0,540. Il y a très peu de différence car les calculs réalisés par chaque thread ne sont pas très compliqués et il faut également compter le temps de création et de regroupement (*join*) des threads.

1.2.3) Parallélisation de createRandomGraph

La fonction de création de graphe présente deux boucles. La première vient simplement allouer des espaces dans un tableau : paralléliser cette boucle ne pose donc aucun problème, il n’y aura pas de concurrence entre les threads.

La deuxième boucle s’occupe de créer le graphe avec des valeurs aléatoires. Il est possible de créer une fonction pour générer ces nombres de manière thread-safe pour permettre à chaque thread de “reprendre” la génération à un moment précis ; cela n’est possible que parce que la fonction de génération de base est simple. Cependant, la boucle imbriquée est exécutée un nombre aléatoire de fois : il n’est pas possible d’assigner un thread à un nombre fixe de tour de boucle. La deuxième boucle de createRandomGraph n’est donc pas parallélisable sans changer la manière dont sont créés les graphes, ce qui pourrait impacter le résultat final.

En suivant le même référentiel, le programme donne le bon résultat avec une moyenne de 18,257 secondes et un écart-type de 0,433. Encore une fois, la boucle parallélisée ne traite pas des calculs très complexes ce qui n’impacte que très peu le temps d’exécution.

1.2.4) Parallélisation de Dijkstra

La fonction de Dijkstra est parallélisable à plusieurs endroits. Les deux premières boucles qui initialisent les différents tableaux utilisés par l’algorithme peuvent être réalisées par plusieurs threads, il n’y a pas de concurrence.

La boucle principale de l’algorithme ne peut cependant pas être parallélisée en tant que telle pour la raison indiquée dans la partie 1.1 : la logique de l’algorithme est intrinsèquement séquentielle. Mais il est toujours possible de paralléliser les deux boucles imbriquées de la fonction.

La première est une réduction avec la fonction minimum : chaque thread cherche un minimum “local” et le minimum de ces minimums est celui qui sera utilisé par la suite de l’algorithme.

La deuxième boucle imbriquée peut être parallélisée sans problème : chaque thread accède à un indice de tableau qu'il calcule par rapport à son propre tour de boucle et qui lui est donc propre ; il n'y aura pas de concurrence.

En suivant le même référentiel, le programme donne le bon résultat avec une moyenne de 4,581 secondes et un écart-type de 0,211 : l'accélération est maintenant nette.

Cette version, où tout partie pouvant être parallélisée l'est, sera nommée "version 1" dans la suite du rapport.

1.2.5) Parallélisation de Dijkstra uniquement

Jusqu'à présent, les différentes parallélisations étaient ajoutées les unes aux autres. Mais, comme montré lors des tests et des profilages, la parallélisation de la fonction createRandomGraph n'accélère pas énormément le programme. Il en est de même pour les deux premières boucles de la fonction Dijkstra : elles ne réalisent pas beaucoup de calculs et le temps pour créer et regrouper les threads est trop important pour qu'une différence se remarque.

En ne parallélisant que la boucle principale de Dijkstra et en utilisant le même référentiel, le programme donne le bon code avec une moyenne de 4,466 secondes d'exécution et un écart-type de 0,194 : c'est légèrement plus rapide que lorsque tout le programme est parallélisé. Cependant, avec des valeurs d'entrées plus grandes, il est possible que le code entièrement parallélisé devienne nettement plus rapide que la version décrite actuellement.

Cette version sera nommée "version 2" dans la suite du rapport.

1.2.6) Parallélisation bizarre de Dijkstra

Au tout début des tests, la boucle principale de la fonction Dijkstra avait été parallélisée de cette manière :

```
#pragma omp parallel for shared(distances, visited)
```

Cette méthode n'est pas sensée fonctionner : chaque thread s'occupe d'une partie du graphe et casse la logique séquentielle de l'algorithme. Cependant, avec différentes entrées, le programme donnait le bon résultat. Il est possible que les premiers threads aient compensé les erreurs des suivants.

En utilisant le même référentiel, le programme s'exécute avec une moyenne de 4,110 secondes et un écart-type de 0,150 : ce qui est plus rapide que les versions parallélisées !

L'exécution n'étant pas prévisible, cette version ne sera donc pas retenue pour la suite.

1.2.7) Changement de logique pour la création de graphe

Comme expliqué précédemment, la fonction createRandomGraph possède une boucle qui n'est pas parallélisable. Cependant, en enlevant les tours de boucle aléatoires et donc en fixant le nombre d'arêtes à chaque exécution, il peut être intéressant de voir si les résultats sont différents et si le programme est plus ou moins lent.

En se basant sur la version 1, en appliquant les modifications précédemment indiquées, et en utilisant le même référentiel, le programme donne le bon résultat (coup de

bol !) et s’exécute avec une moyenne de 4,595 secondes et un écart-type de 0,633 : ce qui est proche des versions 1 et 2.

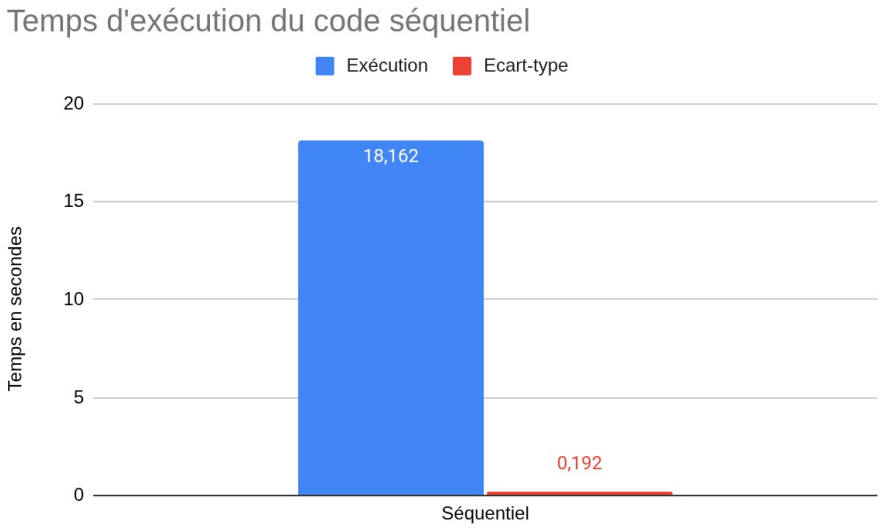
Cette version sera nommée “version 3” dans la suite du rapport, mais ne sera prise en compte que si les résultats qu’elle donne sont identiques aux autres versions.

1.3) Statistiques et analyse

Pour rappel, les tests précédents étaient réalisés avec 16 threads, 200 000 sommets, 100 000 arêtes, une graine de 100, et le flag -O2 lors de la compilation.

Afin de mieux comprendre des différents résultats décrits précédemment, voici deux graphiques résumant les temps moyens d’exécutions avec écart-type :

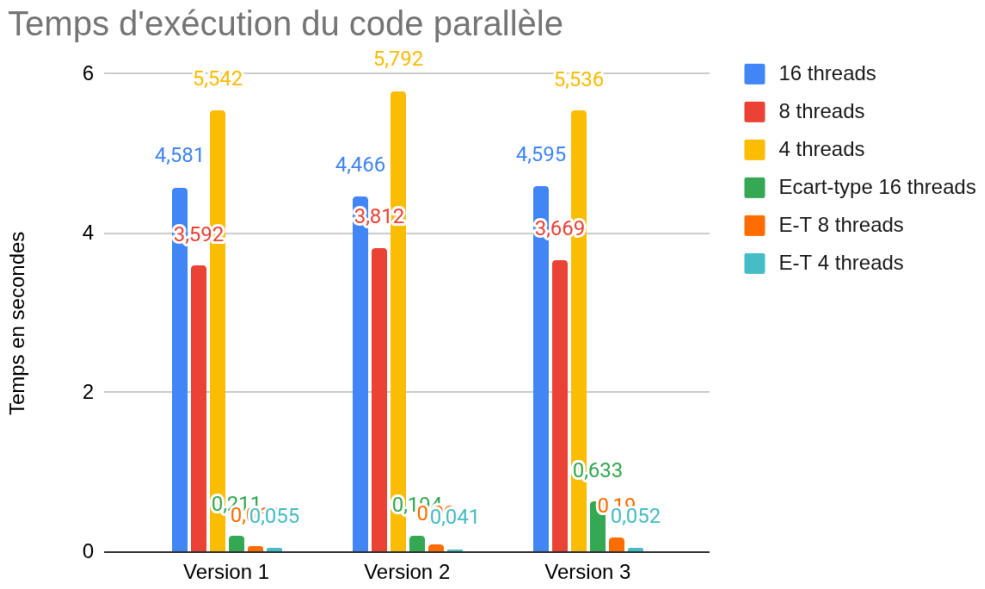
Pour le programme séquentiel :



Version tableau :

Séquentiel	Exécution	Ecart-type
Séquentiel	18,162	0,192

Pour les programmes parallélisés :



Version tableau :

Parallèle	16 threads	8 threads	4 threads	Ecart-type 16 threads	E-T 8 threads	E-T 4 threads
Version 1	4,581	3,592	5,542	0,211	0,065	0,055
Version 2	4,466	3,812	5,792	0,194	0,09	0,041
Version 3	4,595	3,669	5,536	0,633	0,19	0,052

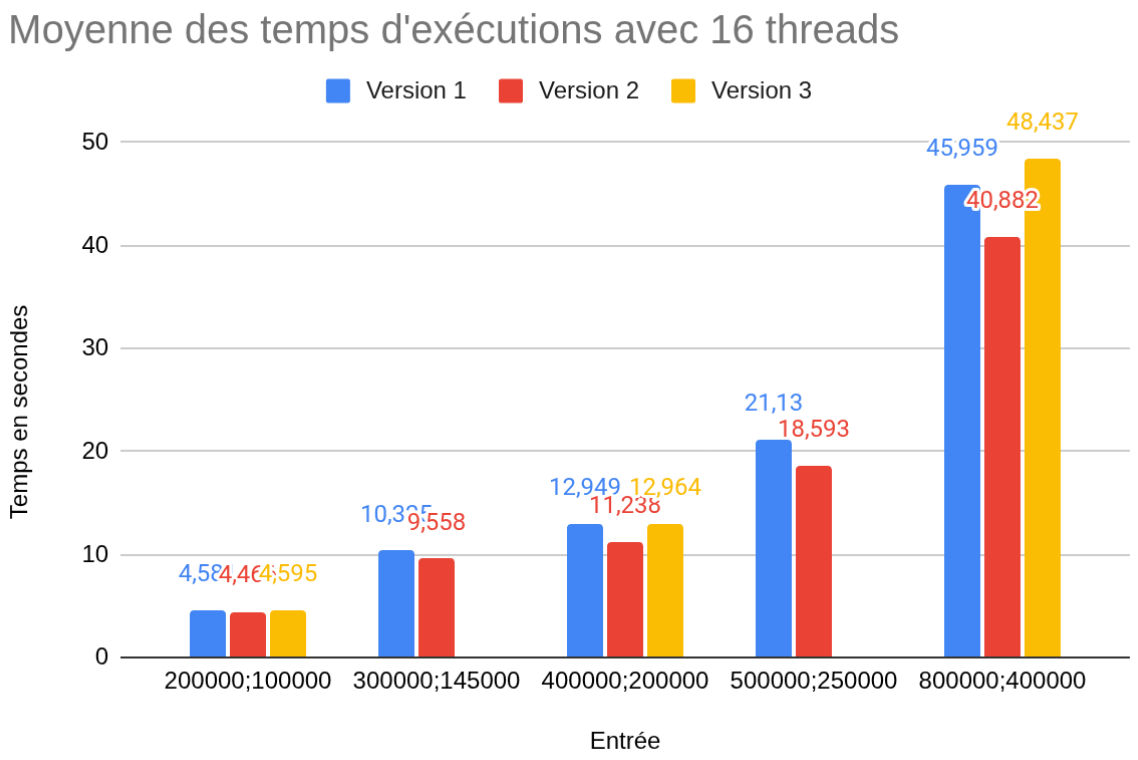
Il est intéressant de noter que le temps d'exécution est plus court avec seulement 8 threads. Cela est probablement dû aux changements de contextes impliqués par la machine qui ne possède que 8 cœurs physiques, à l'attente liée à la création et au regroupement des threads,...

2) Montée en charge

Il a été montré précédemment que les données d'entrées étaient probablement trop petites : les temps d'exécutions moyens des différentes versions parallélisées sont très proches et une version ne se démarque pas suffisamment nettement pour affirmer qu'elle est "mieux" (dans l'optique de traiter des grands graphes).

Voici trois graphes représentant les temps moyens d'exécutions avec les données d'entrées (notées : "nombre de sommets ; nombre d'arêtes") des différentes versions en fonction du nombre de threads :

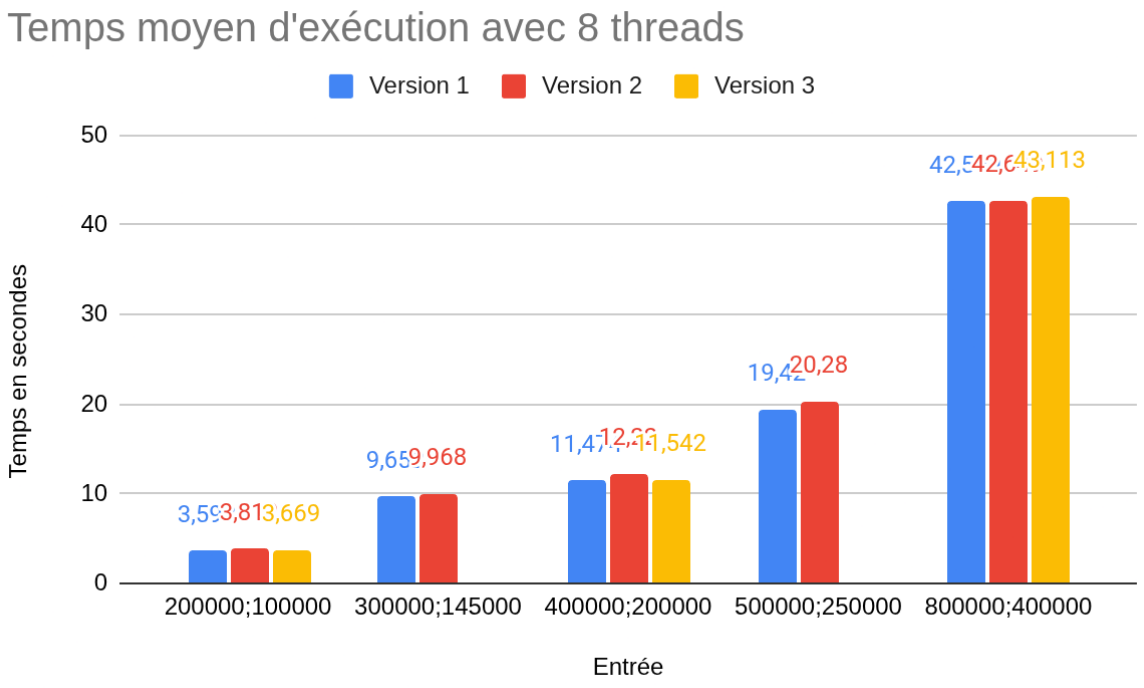
Pour 16 threads :



Version tableau :

Entrée	Version 1	Version 2	Version 3
200000;100000	4,581	4,466	4,595
300000;145000	10,325	9,558	
400000;200000	12,949	11,238	12,964
500000;250000	21,13	18,593	
800000;400000	45,959	40,882	48,437

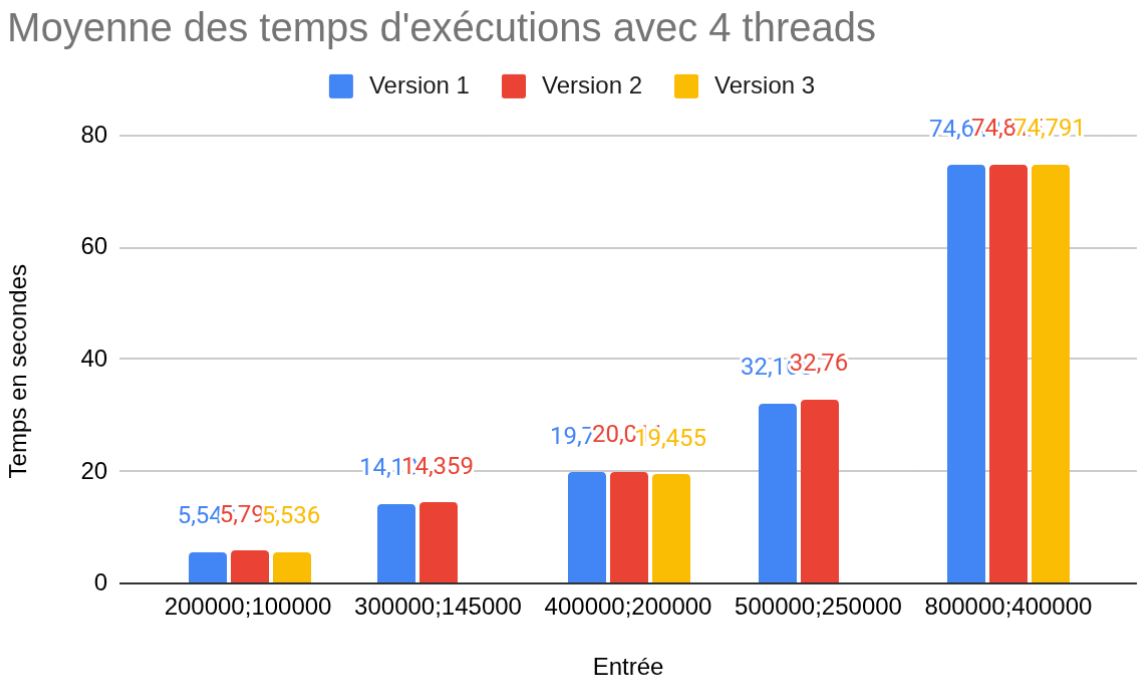
Pour 8 threads :



Version tableau :

Entrée	Version 1	Version 2	Version 3
200000;100000	3,592	3,812	3,669
300000;145000	9,652	9,968	
400000;200000	11,474	12,22	11,542
500000;250000	19,42	20,28	
800000;400000	42,599	42,649	43,113

Pour 4 threads :



Version tableau :

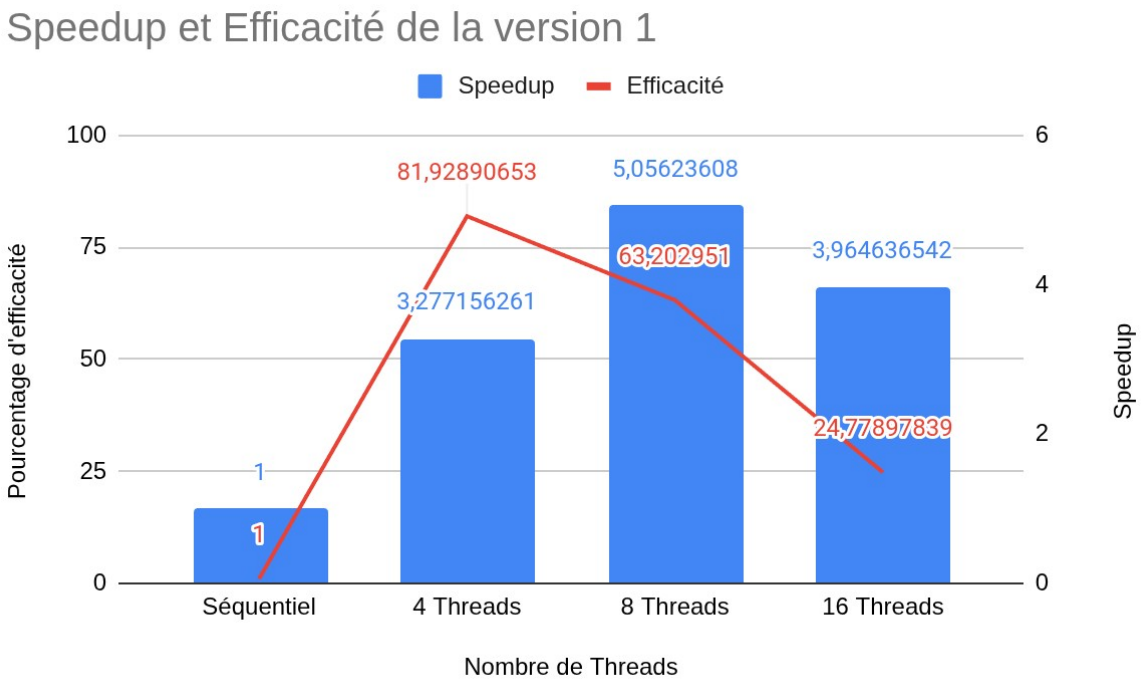
Entrée	Version 1	Version 2	Version 3
200000;100000	5,542	5,792	5,536
300000;145000	14,104	14,359	
400000;200000	19,712	20,011	19,455
500000;250000	32,165	32,76	
800000;400000	74,643	74,815	74,791

Il est à noter que quand des données sont manquantes, cela indique que la version 3 n’a pas donné le résultat correct. Ce comportement avait été décrit dans la section 1.2.7.

Grâce à ces 3 graphiques, on peut remarquer que les temps d’exécutions sont relativement proches entre les versions, même entre l’utilisation de 16 ou 8 threads. Une différence commence à apparaître lors de l’exécution avec comme entrée 800 000 sommets et 400 000 arêtes : la version 2 semble plus rapide avec 16 threads. Il aurait fallu tester avec des valeurs encore plus grandes pour voir si cette tendance continuait, cependant l’ordinateur sur lequel les tests étaient réalisés ne supportait pas de telles exécutions.

Avant de réaliser une conclusion, il peut aussi être intéressant de regarder le Speedup et l'Efficacité des deux premières versions (la troisième ne donnant pas toujours les bons résultats, elle est à présent écartée). Voici deux graphiques décrivant ces données :

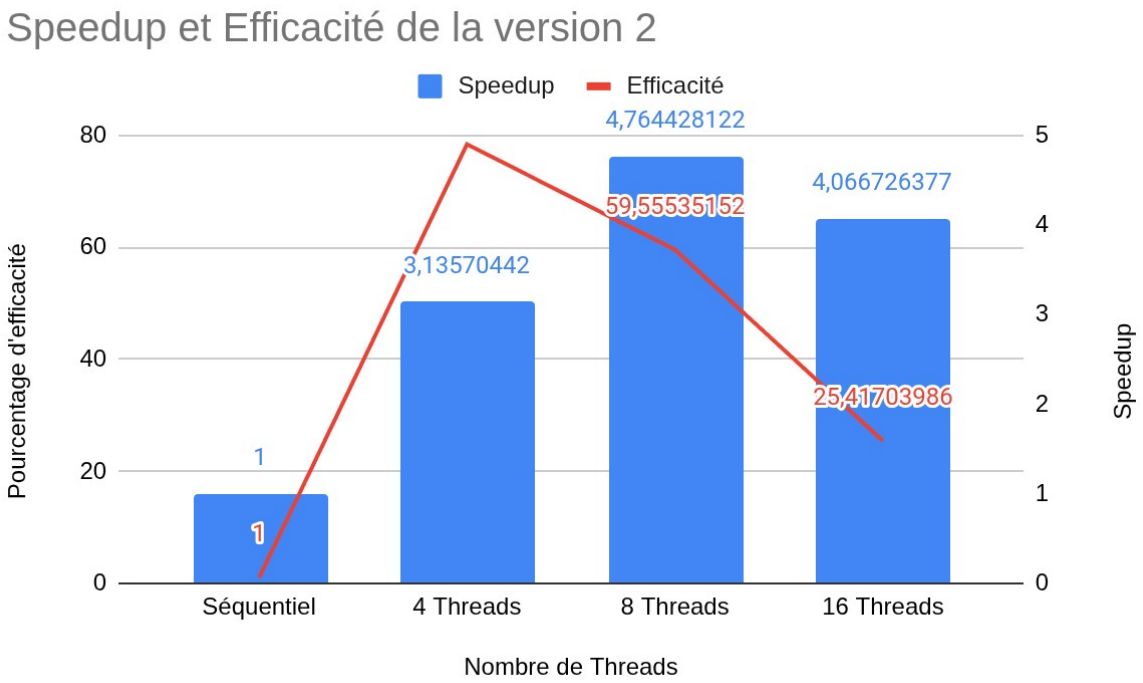
Pour la version 1 :



Version tableau :

Nombre de Threads	Speedup	Efficacité
Séquentiel	1	1
4 Threads	3,277156261	81,92890653
8 Threads	5,05623608	63,202951
16 Threads	3,964636542	24,77897839

Pour la version 2 :



Version tableau :

Nombre de Threads	Speedup	Efficacité
Séquentiel	1	1
4 Threads	3,13570442	78,3926105
8 Threads	4,764428122	59,55535152
16 Threads	4,066726377	25,41703986

Encore une fois, les valeurs sont très proches pour les deux versions. De manière générale, on remarque que l'utilisation de 8 threads semble "mieux" car elle présente le plus de rapidité par rapport à l'efficacité.

3) Conclusion

Avec toutes les données récoltées au cours des différents tests, il est possible de tirer plusieurs conclusions.

Premièrement, d'un point de vue rapidité par rapport au code séquentiel, paralléliser tout ce qui est parallélisable ou uniquement la boucle principale de la fonction de Dijkstra ne change pas drastiquement les temps moyens d'exécutions.

Deuxièmement, pour de très grandes valeurs d'entrées, il semble que la version 2 soit plus rapide (ce qui avait été prédit lors du profilage du code séquentiel) : l'overhead de communication reste trop coûteux pour que la parallélisation complète soit profitable.

S'il fallait ne choisir qu'une seule version, il serait donc plus efficace d'utiliser la version 2 avec 8 threads pour des entrées inférieures à 400 000 sommets et 200 000 arêtes, et 16 threads si les entrées sont plus grandes.

3.1) Améliorations possibles

Pour compléter l’étude, il aurait été intéressant d’utiliser OpenACC en plaçant l’une ou les deux fonctions dans le GPU et voir quelle version est plus efficace. Cela devrait être possible étant donné qu’un graphe est représenté par plusieurs tableaux et que les actions réalisées sur ces tableaux ne sont pas très complexes.

Aussi, l'algorithme de Dijkstra n’étant pas complètement parallélisable, il aurait été possible de le changer pour un autre adapté à la recherche du plus court chemin en environnement parallélisé, comme l’algorithme du Delta-Stepping.